Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Program: Software Engineering

Operating Systems

Laboratory Work No. 7

Three Small Programs: Processes, Redirection, a Pipe

Track A: The Real OS, from the Outside (Linux)

Elaborated by:

st. gr. FAF-241
Luchiciov Alexei

Verified by:

Reiţman Patricia

Chișinău – 2026

Contents

1. Introduction . . . 3
2. fork + exec + wait . . . 3
2.1. Implementation . . . 3
2.2. Execution and Modified Commands . . . 3
2.3. Observations . . . 4
3. A Zombie Process . . . 4
3.1. Experiment . . . 4
3.2. Observations . . . 4
4. Output Redirection . . . 5
4.1. Implementation . . . 5
4.2. Execution . . . 5
4.3. Observations . . . 5
5. A Pipe: ls /etc | wc -l . . . 5
5.1. Implementation . . . 6
5.2. Comparison and Intentional Failure . . . 6
5.3. Observations . . . 6
6. Optional: Three-stage Pipeline . . . 7
6.1. Implementation . . . 7
6.2. Testing . . . 7
7. Conclusion . . . 7

1. Introduction

The purpose of this laboratory work was to examine how Linux creates processes, redirects program output, and connects programs through pipes. I implemented and tested four small C programs: **run.c**, **zombie.c**, **redirect.c**, and **pipeline.c**. I also completed the optional three-stage pipeline in **pipeline3.c**.

All experiments were performed in an Ubuntu virtual machine, in the **~/os-lab7** directory. The compiler was GCC 15.2.0. The output of each experiment was saved to a text file, and the observations below are based on the actual terminal results.

2. fork + exec + wait

The first program demonstrates the three steps used by a shell to execute a command: **fork()** creates a child, **execlp()** replaces the child with another program, and **waitpid()** allows the parent to collect the child's exit status.

2.1. Implementation

```
pid_t pid = fork();
if (pid < 0) { perror("fork"); return 1; }

if (pid == 0) {
    printf("child: my PID is %d, my parent is %d\n", getpid(), getppid());
    fflush(stdout);
    execlp("ls", "ls", "-l", (char *)NULL);
    perror("exec");
    exit(127);
}

int status;
waitpid(pid, &status, 0);
```

The program printed the PIDs of the parent and child before executing **ls -l**. Next, I modified the **execlp()** call to run **date**, and then to run a nonexistent program named **nosuchprog**.

2.2. Execution and Modified Commands

```
$ gcc -Wall -Wextra -o run run.c
$ ./run
parent: my PID is 4753
child: my PID is 4755, my parent is 4753
total 24
-rwxrwxr-x 1 alex_007m alex_007m 16392 Nov  9 12:22 run
-rw-rw-r-- 1 alex_007m alex_007m   662 Nov  9 12:22 run.c
parent: child 4755 ended with status 0

$ # execlp("date", "date", (char *)NULL)
$ ./run
parent: my PID is 4837
child: my PID is 4839, my parent is 4837
Mon Nov  9 12:22:49 PM UTC 2026
parent: child 4839 ended with status 0

$ # execlp("nosuchprog", "nosuchprog", (char *)NULL)
$ ./run
parent: my PID is 4855
child: my PID is 4857, my parent is 4855
exec: No such file or directory
parent: child 4857 ended with status 127
```

After testing the two modifications, I restored **run.c** to the original version using **ls -l**. The successful executions returned status 0, while the failed execution returned 127.

2.3. Observations

1. **Why is perror("exec") printed only with nosuchprog?** A successful **execlp()** replaces the child process image and never returns to the next line. The program **nosuchprog** does not exist, so **execlp()** returns an error and the child prints the message using **perror()**.

2. **What is the child's parent PID?** For the first run, the child printed **my parent is 4753**. This is exactly the PID printed by the parent process; the child itself had PID 4755.

3. **What exit statuses were observed?** With **ls**, the parent observed status **0**. With **nosuchprog**, it observed **127**, because the child explicitly called **exit(127)** after the failed exec call.

3. A Zombie Process

In the second task I copied the original **run.c** to **zombie.c** and inserted **sleep(20)** immediately before **waitpid()**. This makes the parent stay alive without immediately collecting the completed child process, temporarily creating a zombie.

3.1. Experiment

```
int status;
sleep(20);
waitpid(pid, &status, 0);

$ gcc -Wall -Wextra -o zombie zombie.c
$ ./zombie > /dev/null &
[1] 4916
$ sleep 2
$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
   4306    4187 Ss   /usr/bin/bash
   4916    4306 S    ./zombie
   4918    4916 Z    [ls] <defunct>
   4921    4306 R+   ps -o pid,ppid,stat,cmd
   4922    4306 S+   tee part2-before.txt
```

The process table showed **./zombie** as PID 4916 and the completed child **[ls] <defunct>** as PID 4918 with PPID 4916. After the parent had slept long enough, I repeated the process-table check.

```
$ sleep 21
$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
   4306    4187 Ss   /usr/bin/bash
   4961    4306 R+   ps -o pid,ppid,stat,cmd
   4962    4306 S+   tee part2-after.txt
```

3.2. Observations

1. **Which letter appears in the STAT column?** The child **ls** had state **Z**, meaning a zombie: it had finished but its parent had not yet collected the exit status. Its command appeared as **[ls] <defunct>**.

2. **Was the zombie still present after 20 seconds?** No. The **[ls] <defunct>** line was absent from the later process table. The call **waitpid(pid, &status, 0)** allowed the parent to reap the child and remove its zombie entry.

4. Output Redirection

The third program reproduces the behavior of **ls -l > out.txt** without using shell redirection inside the program. In the child, file descriptor 1 (standard output) is closed, and **open()** creates or truncates **out.txt**. Since descriptor 1 becomes the smallest unused descriptor, **open()** receives it.

4.1. Implementation

```
if (pid == 0) {
    close(1);
    int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); exit(1); }
    fprintf(stderr, "open gave descriptor %d\n", fd);
    execlp("ls", "ls", "-l", (char *)NULL);
    perror("exec");
    exit(127);
}
```

4.2. Execution

```
$ gcc -Wall -Wextra -o redirect redirect.c
$ ./redirect
open gave descriptor 1
done: the listing is in out.txt
$ head -3 out.txt
total 92
-rw-rw-r-- 1 alex_007m alex_007m     0 Nov  9 12:24 out.txt
-rw-rw-r-- 1 alex_007m alex_007m   135 Nov  9 12:22 part1-date.txt
```

The terminal only displayed the diagnostic message sent to **stderr** and the parent's completion message. The listing produced by **ls** was stored inside **out.txt**. The captured listing began with **total 92**.

4.3. Observations

1. **Which descriptor did open() return, and why?** It returned descriptor **1**. The child called **close(1)** first, so the next **open()** call reused the smallest available descriptor.

2. **Why did the ls listing not appear on screen?** Standard output now referred to **out.txt** rather than the terminal. The **ls** program continued writing to descriptor 1, but those bytes went into the file.

3. **Which descriptor should be closed to redirect errors?** Descriptor **2**, which is the standard error stream (**stderr**), should be closed and then replaced with a file descriptor for the error-output file.

5. A Pipe: ls /etc | wc -l

In this task I implemented the shell pipeline **ls /etc | wc -l** using one pipe and two child processes. The first child sends the output of **ls** into the write end of the pipe, and the second child reads from the other end and counts the incoming lines.

5.1. Implementation

```
int p[2];
pipe(p);

if (fork() == 0) {              /* first child: ls */
    close(1); dup(p[1]);
    close(p[0]); close(p[1]);
    execlp("ls", "ls", "/etc", (char *)NULL);
}
if (fork() == 0) {              /* second child: wc */
    close(0); dup(p[0]);
    close(p[0]); close(p[1]);
    execlp("wc", "wc", "-l", (char *)NULL);
}
close(p[0]);
close(p[1]);
wait(NULL); wait(NULL);
```

Closing the unused ends of the pipe is essential. After both children are created, the parent must close both **p[0]** and **p[1]** so that the reader can receive an end-of-file notification when the writer exits.

5.2. Comparison and Intentional Failure

```
$ gcc -Wall -Wextra -o pipeline pipeline.c
$ ./pipeline
236
$ ls /etc | wc -l
236
```

Both implementations printed **236**, confirming that the custom pipeline matched the equivalent shell command in the virtual machine.

```
/* Modified parent code (intentional test) */
close(p[0]);
/* close(p[1]); */
wait(NULL);
wait(NULL);

$ ./pipeline
[program did not terminate; stopped with Ctrl+C]
```

As required by the exercise, I temporarily commented out the parent's **close(p[1])** call. The program did not terminate, and I stopped it with **Ctrl+C**. After restoring the statement and recompiling, **./pipeline** successfully printed **236** again.

5.3. Observations

1. **Did the custom program and the shell command agree?** Yes. Both **./pipeline** and **ls /etc | wc -l** printed **236**.
2. **Which child failed to terminate after the modification?** The **wc** child continued waiting. The **ls** child finished sending its output.
3. **What was wc waiting for?** The reader was waiting for end-of-file (**EOF**). Because the parent kept the pipe's write end **p[1]** open, the reader never saw EOF.

6. Optional: Three-stage Pipeline

I also implemented the optional program **pipeline3.c**, corresponding to **ls /usr/bin | grep x | wc -l**. It creates two pipes and three child processes. The first child lists programs from **/usr/bin**, the second filters lines containing the letter **x**, and the third counts the remaining lines.

6.1. Implementation

```
/* First child: output goes into the first pipe. */
dup2(p1[1], STDOUT_FILENO);    /* ls /usr/bin */

/* Second child: between the first and second pipes. */
dup2(p1[0], STDIN_FILENO);
dup2(p2[1], STDOUT_FILENO);    /* grep x */

/* Third child: input comes from the second pipe. */
dup2(p2[0], STDIN_FILENO);     /* wc -l */
```

Each child closed all unused pipe descriptors after **dup2()**, and the parent closed all four pipe ends before waiting for the three children.

6.2. Testing

```
$ gcc -Wall -Wextra -o pipeline3 pipeline3.c
$ ./pipeline3
229
$ ls /usr/bin | grep x | wc -l
229
```

Both the custom three-stage pipeline and the direct shell pipeline returned **229**, so the optional task was successfully verified.

7. Conclusion

In this laboratory work I studied Linux process creation, execution, and synchronization through **fork()**, **execlp()**, and **waitpid()**. The first program showed how a child can run another command and how an unsuccessful exec call produces exit status 127. The second experiment demonstrated a zombie process in the process table and showed that the parent removes it by calling **waitpid()**.

I also tested how file descriptors determine where output is written. By reusing descriptor 1, I redirected the output of **ls** to **out.txt**. Finally, I constructed two-stage and three-stage pipelines using **pipe()**, **fork()**, and descriptor duplication. The pipeline outputs matched the native shell commands: **236** and **229**, respectively. The intentional deadlock-like waiting experiment showed why unused pipe ends must be closed correctly.

All four required C programs and the optional three-stage pipeline were compiled and tested in the Ubuntu virtual machine. The experiment results and observations demonstrate the basic process-management and interprocess-communication mechanisms of Linux.